

Skin Type Detection and Skincare Recommendation using Deep Learning Model DenseNet169

*Project report submitted to Amrita Vishwa Vidyapeetham in partial fulfilment of the requirement
for the Degree of*

BACHELOR of TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

submitted by

Pranathi M
Lasya Kapuluru
Varnika Sood

AM.EN.U4CSE21135
AM.EN.U4CSE21169
AM.EN.U4CSE21175



AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM
(Estd. U/S 3 of the UGC Act 1956)

AMRITAPURI CAMPUS

KOLLAM – 690525

JUNE 2024

CONTENTS

S. No.	Title	Page No.
1	Abstract	3
2	Communication	4
3	Planning	5
4	Modelling	7
5	Coding	18
6	Testing	21
7	Conclusion	26
8	References	27

ABSTRACT

The exponential growth of the skincare industry underscores the need for personalized skincare solutions, which can be effectively addressed through advanced technological methods. This study presents an innovative approach to skin type detection and personalized skincare recommendation using the DenseNet169 deep learning model. DenseNet169, a state-of-the-art convolutional neural network known for its efficiency in feature propagation and parameter efficiency, serves as the backbone of the skin type classification system. The model was trained on a diverse dataset encompassing various skin types, ensuring robustness and accuracy in real-world applications.

Following the classification, a comprehensive web scraping strategy was implemented to collect detailed information on a wide range of skincare products. This data was used to build a recommender system that aligns skincare products with individual skin types, enhancing the personalization of skincare routines. The recommender system leverages collaborative filtering and content-based filtering techniques to provide users with optimal product suggestions tailored to their specific skin needs.

Extensive testing and validation of the model were conducted to ensure high precision and recall in skin type detection, as well as reliability in product recommendations. User feedback was incorporated to refine the system further, enhancing user satisfaction and engagement. This study not only demonstrates the effectiveness of DenseNet169 in dermatological applications but also bridges the gap between machine learning and consumer health by offering a practical tool for personalized skincare. The findings highlight the potential of deep learning models in transforming skincare practices, paving the way for future advancements in personalized dermatological care.

COMMUNICATION

WHO IS YOUR CLIENT?

The clients for this project are a leading skincare products manufacturer. They are seeking to improve consumer satisfaction and skincare efficacy through innovative technological solutions. Their focus is on addressing challenges in the skincare industry related to product effectiveness and personalized skincare recommendations based on individual skin types and conditions. These clients are interested in leveraging advanced technologies, such as deep learning and image analysis, to revolutionize how skincare products are recommended and used by consumers.

MEETING:

Participants: Mallampati Pranathi, Lasya K, Varnika Sood, Ms. Anjali.T

Meeting minutes: 45min – 1hr

Key points:

- Overview of the current challenges in the skincare industry.
- Understanding the client's strategic priorities and technological aspirations.
- Discussion on how the deep learning technologies can enhance skincare product recommendations and user experience.
- Defining the scope of the project.
- Agreement on the next steps for project initiation and development.
- Discussion on timelines, milestones, and deliverables.

PROBLEM STATEMENT

The communication phase outlines the critical issues faced by skincare consumers and sets the stage for the project's objective which is to develop an intelligent system that bridges the gap between skincare analysis and product recommendation. By understanding the client's perspective, meeting outcomes, and defining the problem statement clearly, our report lays the groundwork for proposing an effective solution that meets market demands and consumer expectations in the skincare industry.

PLANNING

Methodology:

1. **User Research:** Conduct comprehensive user research to understand prevalent skincare concerns, preferences, and challenges in selecting skincare products. This research will guide the development of features that effectively address user needs.
2. **Deep Learning Implementation:** Implement sophisticated deep learning techniques, such as convolutional neural networks (CNNs), to develop robust skin type classification models. These models will analyze facial features extracted from uploaded images to accurately categorize skin types.
3. **Testing and Validation:** Employ rigorous testing methodologies, including unit testing to validate individual components of the classification models, and integration testing to ensure seamless functionality across different modules of the system. User acceptance testing will verify that the software meets user expectations and performs accurately.
4. **User Feedback Loop:** Establish mechanisms for continuous user feedback collection. This feedback will be instrumental in iteratively refining the software based on user insights, ensuring ongoing improvement and increasing user satisfaction.

Team Roles and Contributions:

Throughout the project, each team member played a distinct role contributing to the successful completion of various phases:

- **Pranathi** served as the Team Lead, actively guiding discussions, providing input on data sources, implementation approaches, and overseeing project scope and objectives.
- **Lasya** assumed the role of Business Analyst, focusing on defining the project scope, suggesting ideas for predictive modelling, and leading the formulation of the problem statement and innovation aspects.
- **Varnika** acted as the Scrum Master, leading discussions on project scope and objectives, facilitating brainstorming sessions, and overseeing feature engineering efforts, including data collection, preprocessing, and initial model selection.

Each team member actively participated in their respective roles, contributing expertise and collaborative effort to achieve project milestones.

Weekly Plan:

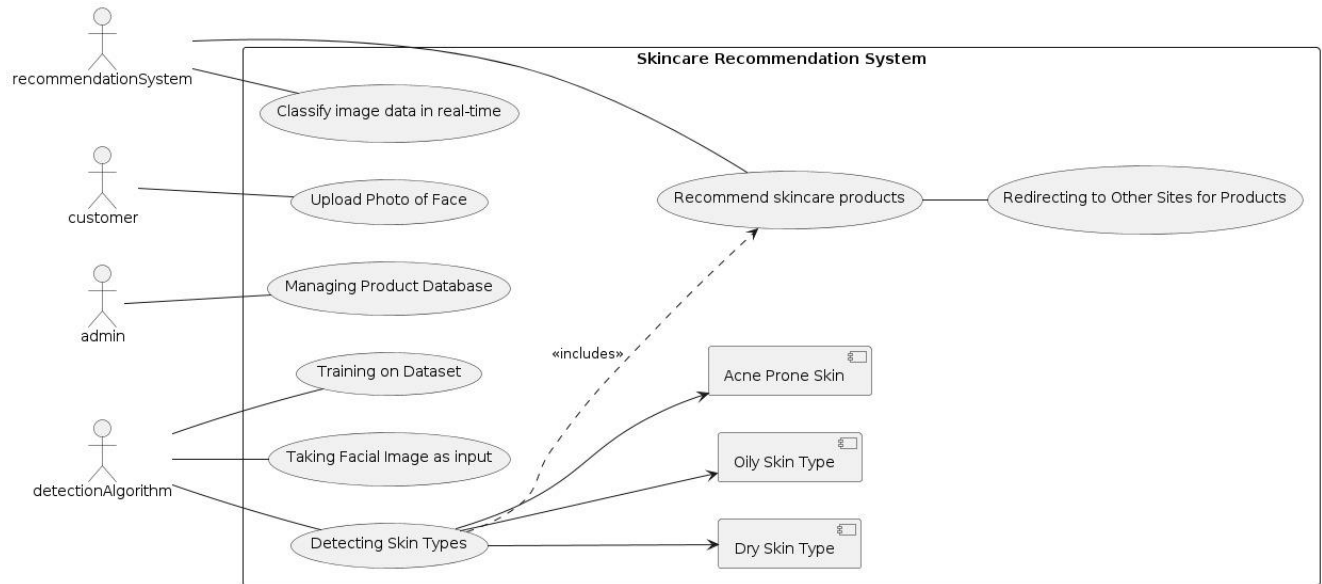
- Week 1 (March 1 – 7): Conduct user research and finalize requirements.
- Week 2 (March 8 - 14): Design the deep learning model architecture.
- Week 3 (March 15 - 21): Collect and preprocess data for training.
- Week 4 (March 22 – 28): Train the initial version of the deep learning model.
- Week 5 (March 29 - April 4): Test and validate the deep learning model.
- Week 6 (April 5 - 11): Fine-tune the model and improve performance.
- Week 7 (April 12 - 18): Web scraping for skincare products.
- Week 8 (April 19 - 25): Build the recommender system using the model.
- Week 9 (April 26 - May 2): Integrate the recommender system with product data.
- Week 10 (May 3 - 9): Test the recommender system for accuracy and reliability.
- Week 11 (May 10 - 16): Gather user feedback and make necessary improvements.
- Week 12 (May 17 – 23): Conduct extensive testing and validation of the model.
- Week 13 (May 24 – 30): Further fine-tune the model and recommender system.
- Week 14 (May 31 - June 6): Perform final testing and validation of all components.
- Week 15 (June 7 – 15): Finalize documentation and prepare for project presentation.

Budget:

The project budget encompasses costs associated with data acquisition, software tools, and any external services that may be required. However, due to the availability of open-source tools, free access to necessary software and data, and the voluntary contributions of team members, the project did not incur any costs. Therefore, the budget for this project was nil.

MODELING

I. Use Case Diagram



Actors:

1. **customer**: Represents the end-users who interact with the skincare recommendation system by uploading photos of their faces.
2. **admin**: Represents system administrators who manage and maintain the product database within the skincare recommendation system.
3. **detectionAlgorithm**: Represents an automated algorithm or subsystem responsible for various tasks related to image processing and data analysis in the system.
4. **recommendationSystem**: Represents the subsystem or component responsible for recommending skincare products based on user input and system analysis.

Use Cases:

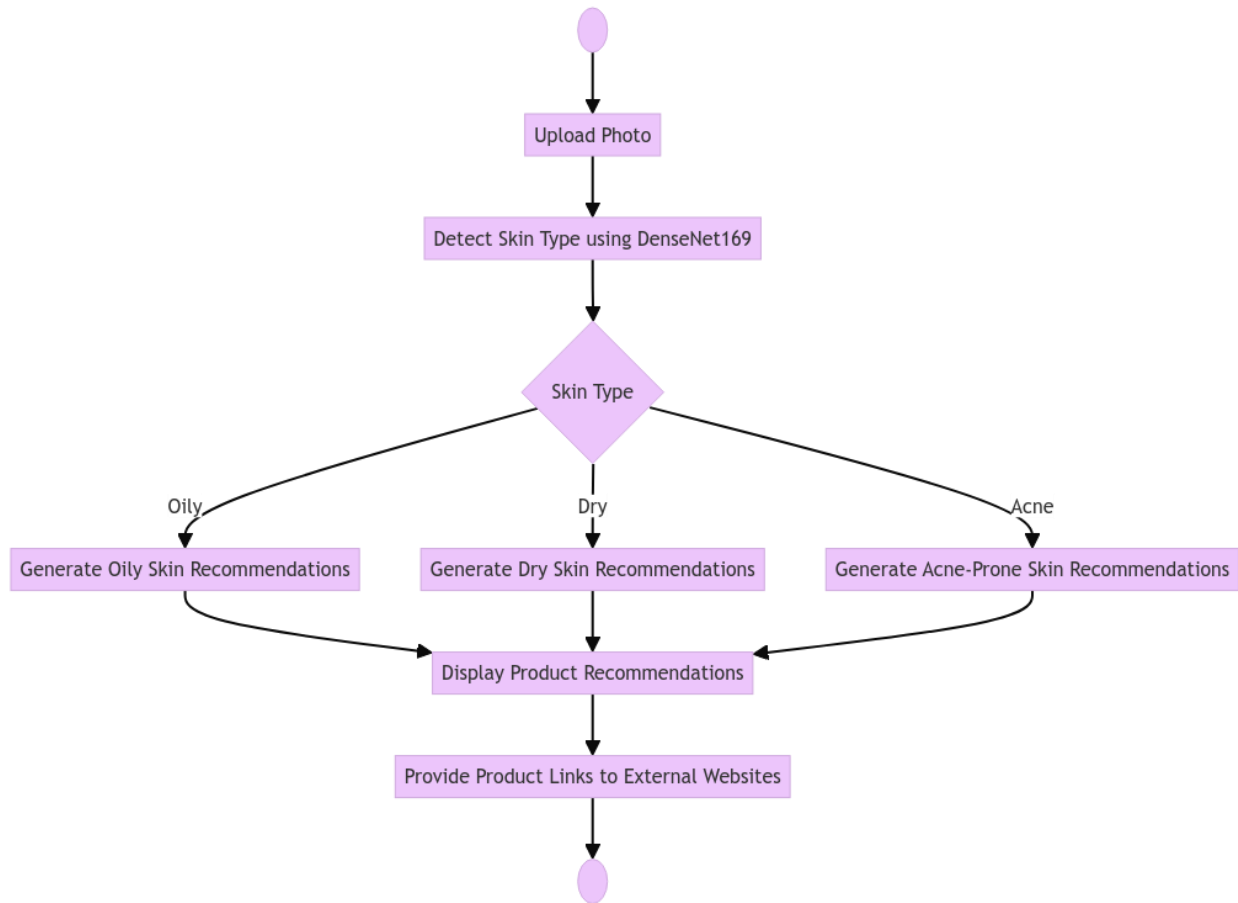
1. **Upload Photo of Face**: Allows customers to upload images of their faces to initiate the skincare analysis process.
2. **Detecting Skin Types**: Involves analyzing uploaded photos to classify the skin type of the customer.
3. **Dry Skin Type, Oily Skin Type, Acne Prone Skin**: Specific classifications of skin types that the system identifies during the detection process.
4. **Classify image data in real-time**: Refers to the process of analyzing image data as it is received in real-time to provide timely feedback.

5. **Recommend skincare products:** Suggests skincare products based on the detected skin type and other relevant factors.
6. **Redirecting to Other Sites for Products:** Possibly involves redirecting customers to external websites or services for purchasing recommended skincare products.
7. **Training on Dataset:** Involves using existing data to train or update the algorithms used in the skincare recommendation system.
8. **Taking Facial Image as input:** Describes the action of capturing facial images directly within the system interface.
9. **Managing Product Database:** Involves administrative tasks related to updating, maintaining, or expanding the database of skincare products.

Relationships:

- **customer** is directly associated with the **Upload Photo of Face** use case, indicating that customers initiate this action.
- **admin** is associated with **Managing Product Database**, illustrating their role in database administration within the system.
- **detectionAlgorithm** is associated with several use cases:
 - **Taking Facial Image as input:** Handling the input of facial images.
 - **Detecting Skin Types:** Performing the classification of skin types.
 - **Training on Dataset:** Training the algorithm based on datasets.
- **recommendationSystem** is associated with:
 - **Classify image data in real-time:** Processing and categorizing image data promptly.
 - **Recommend skincare products:** Generating recommendations based on the processed data.
 - **Redirecting to Other Sites for Products:** Possibly managing redirections for purchasing recommended products.
- **Detecting Skin Types:**
 - **Includes:**
 - **Dry Skin Type, Oily Skin Type, Acne Prone Skin:** These are specific variations or classifications of skin types that are identified during the skin type detection process.
 - **Triggers:**
 - **Recommend skincare products:** This relationship suggests that after identifying the skin type, the system proceeds to recommend appropriate skincare products tailored to that specific skin type.

II. Activity Diagram



Activities and Relationships:

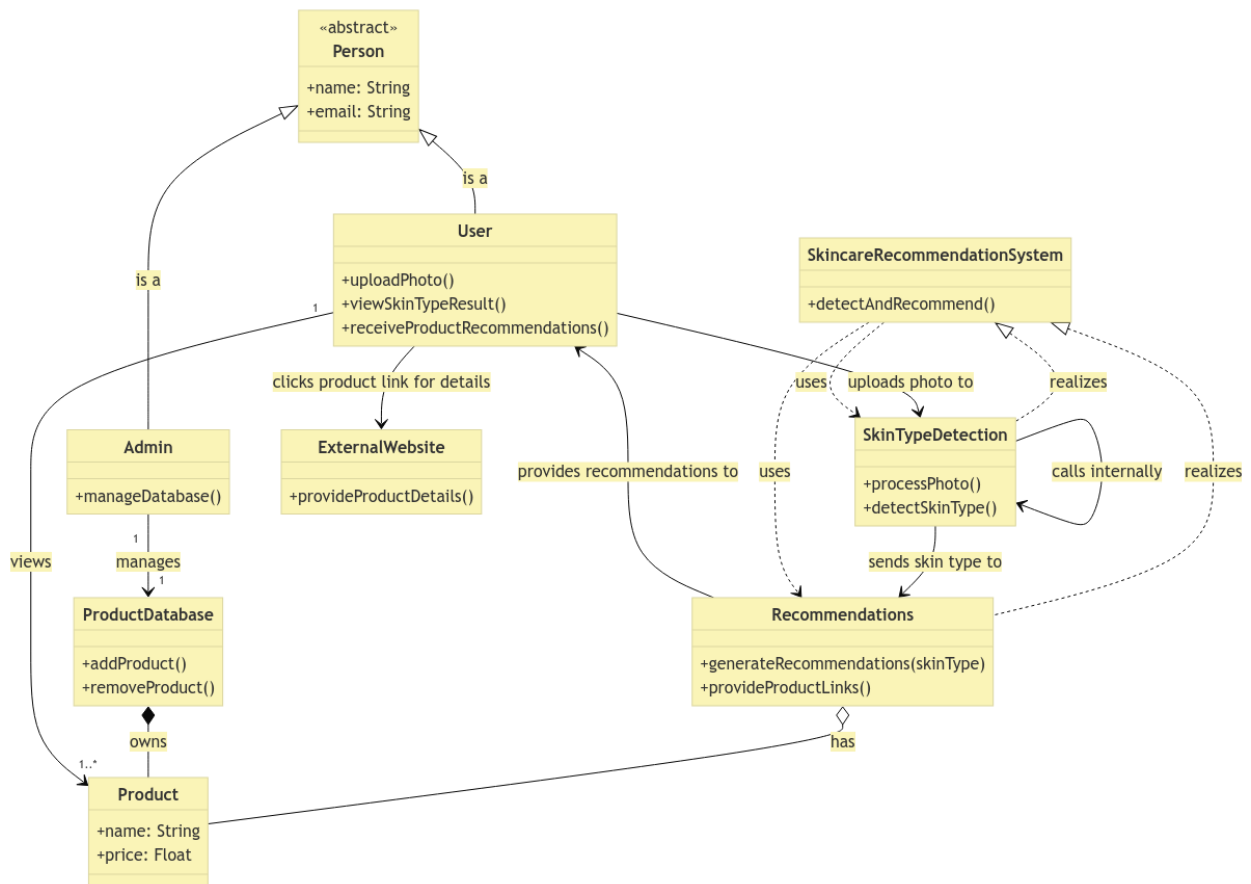
1. **Start:** Represents the beginning of the process.
2. **Upload Photo:** Customers upload a photo of their face to initiate the analysis.
3. **Detect Skin Type using DenseNet169:** The system utilizes the DenseNet169 algorithm to analyze the uploaded photo and detect the skin type.
4. **Skin Type:** Based on the analysis:
 - If the skin type is **Oily**, flow goes to generate oily recommendations.
 - If the skin type is **Dry**, flow goes to generate dry recommendations.
 - If the skin type is **Acne-Prone**, flow goes to generate acne-prone recommendations.
5. **Generate Recommendations:** Depending on the detected skin type:
 - **Oily Skin Recommendations:** Generates recommendations tailored for oily skin.
 - **Dry Skin Recommendations:** Generates recommendations tailored for dry skin.

- **Acne-Prone Skin Recommendations:** Generates recommendations tailored for acne-prone skin.
- 6. **Display Product Recommendations:** After generating recommendations, the system displays them to the user.
- 7. **Provide Product Links to External Websites:** For each recommendation displayed, the system provides links to external websites where users can purchase the recommended products.
- 8. **End:** Represents the completion of the process.

Flow and Relationships:

- **Start** initiates the process.
- **Upload Photo** leads to **Detect Skin Type**.
- **Detect Skin Type** determines the type of skin detected.
- **Skin Type** directs flow based on the detected type (Oily, Dry, Acne-Prone).
- **Generate oily, dry, acne-prone Recommendations** are specific to each skin type detected.
- **Display Product Recommendations** follows after recommendations are generated.
- **Provide Product Links** offers external links for purchasing recommended products.
- **End** signifies the completion of the process.

III. Class Diagram



Classes (and their Responsibilities)

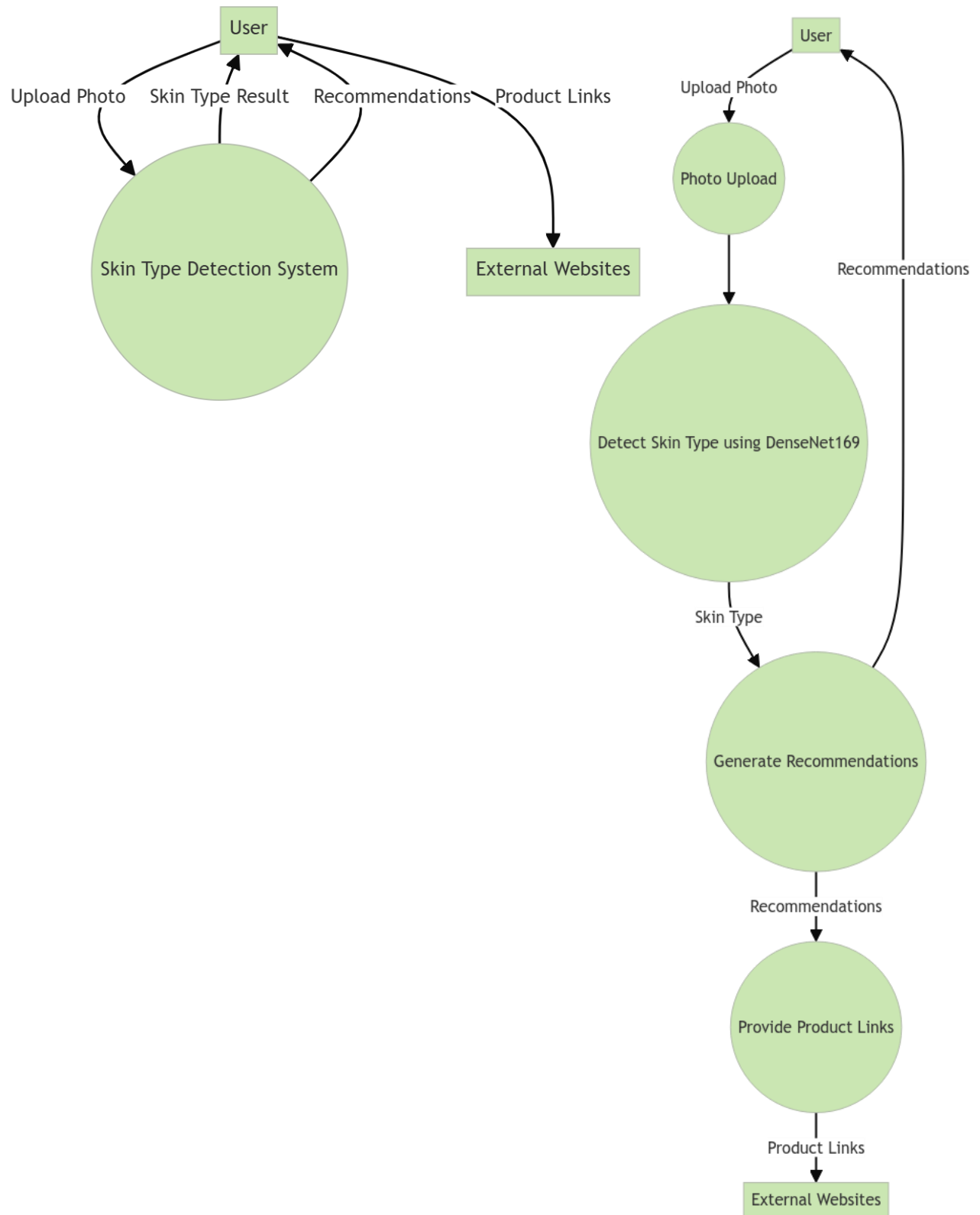
1. **Person** (abstract):
 - Represents individuals with `name` and `email`.
2. **User**:
 - Inherits from `Person`.
 - **Methods:** `uploadPhoto()`, `viewSkinTypeResult()`, `receiveProductRecommendations()`.
3. **Admin**:
 - Inherits from `Person`.
 - **Methods:** `manageDatabase()`.
4. **SkinTypeDetection**:
 - Processes uploaded photos to `detectSkinType()`.
5. **Recommendations**:
 - Generates `generateRecommendations(skinType)` based on detected skin type.

- Provides `provideProductLinks()` to external websites.
- 6. **Product:**
 - Represents skincare products with `name` and `price`.
- 7. **ProductDatabase:**
 - Manages `addProduct()` and `removeProduct()` operations.
- 8. **ExternalWebsite:**
 - Provides `provideProductDetails()` about skincare products.
- 9. **SkincareRecommendationSystem:**
 - Uses `SkinTypeDetection` and `Recommendations` to `detectAndRecommend()` skincare products.

Relationships

- **Inheritance:**
 - `User` and `Admin` inherit from `Person`.
- **Aggregation:**
 - `Recommendations` **uses** `Product`.
- **Composition:**
 - `ProductDatabase` **manages** `Product`.
- **Realization:**
 - `SkincareRecommendationSystem` **implements** `SkinTypeDetection` and `Recommendations`.
- **Associations:**
 - **User** uploads photos to **SkinTypeDetection**.
 - **SkinTypeDetection** sends skin type to **Recommendations**.
 - **Admin** manages **ProductDatabase**.
 - **User** interacts with **ExternalWebsite** for product details.
- **Dependency:**
 - `SkincareRecommendationSystem` **depends on** `SkinTypeDetection` and `Recommendations`.

IV. Data Flow Diagram



DFD 0

- **A (User):** Represents the user interacting with the system.
- **B (Skin Type Detection System):** System component responsible for detecting the skin type based on the uploaded photo.
 - **Upload Photo:** User uploads a photo.
 - **Skin Type Result:** System provides the detected skin type.
 - **Recommendations:** System provides skincare product recommendations based on the detected skin type.
- **C (External Websites):** Represents external websites where users can find more details or purchase recommended products.

Level of Abstraction:

- This flowchart provides a high-level overview of the interaction between the user and the system.
- It focuses on the main steps: uploading a photo, detecting skin type, receiving recommendations, and accessing product links.
- It abstracts away specific details about how skin type detection or recommendation generation is implemented within the system.

DFD 1

- **A (User):** Represents the user.
- **B (Photo Upload):** User uploads a photo.
- **C (Detect Skin Type using DenseNet169):** System uses the DenseNet169 algorithm to detect the skin type based on the uploaded photo.
- **D (Generate Recommendations):** System generates skincare product recommendations based on the detected skin type.
 - **Recommendations:** Indicates the flow returns to the user to provide the recommendations.
- **E (Provide Product Links):** System provides links to external websites where recommended products can be purchased.
- **F (External Websites):** Represents the external websites.

Level of Abstraction:

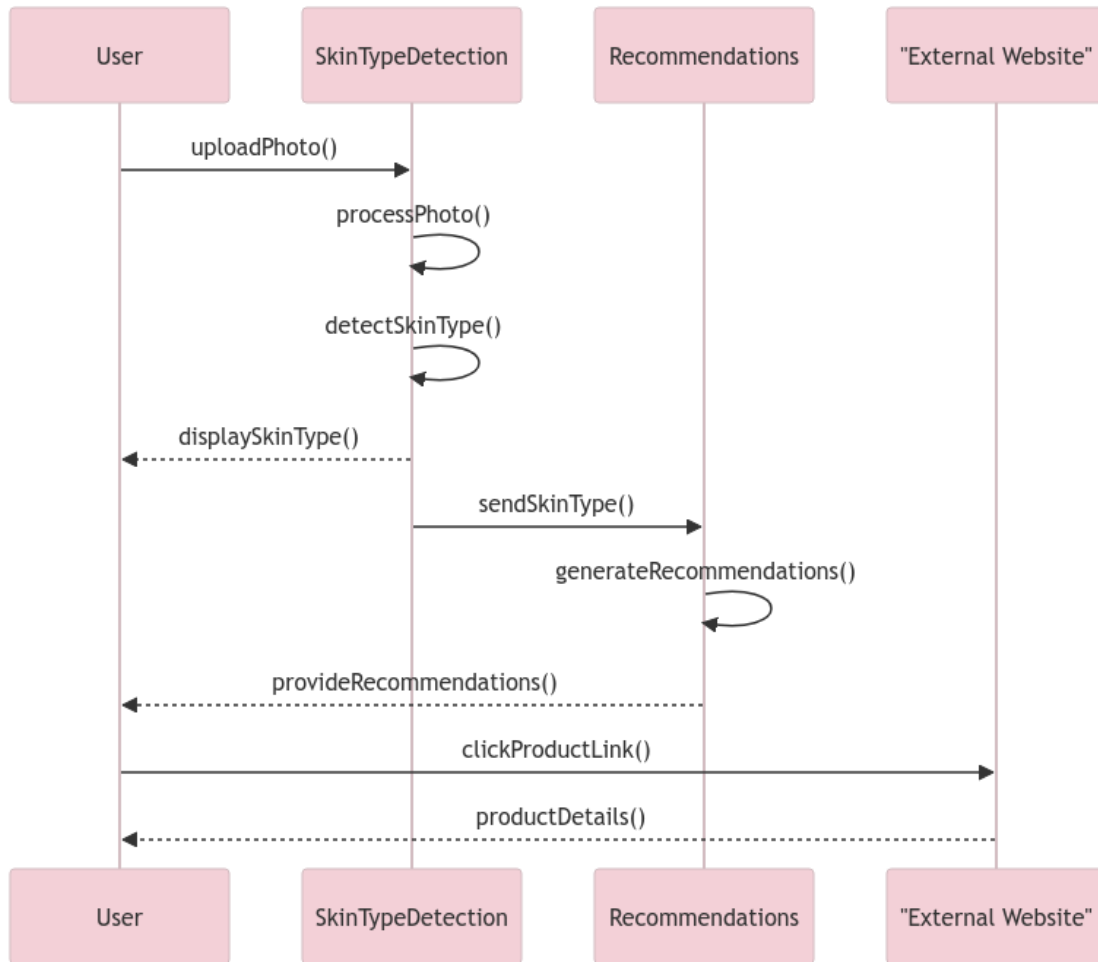
- This flowchart provides a more detailed view compared to the first one.
- It specifies the use of the DenseNet169 algorithm for skin type detection.

- It explicitly shows the flow from photo upload to skin type detection, recommendation generation, and finally, product links.
- It maintains a focused view on the sequential steps involved in the process.

Comparison of Levels of Abstraction:

- **DFD 0** focuses on high-level user interactions and system outputs (skin type detection result, recommendations, product links) without delving into specific implementation details.
- **DFD 1** provides a more detailed sequence of actions, including the specific algorithm used for skin type detection (DenseNet169) and the flow of data from detection to recommendation and external interaction.

V. Sequence Diagram



Module 1: User Interaction

Description: It handles the initial photo upload, skin type detection, and display to the user.

- **User:** Initiates the process by uploading a photo.
- **SkinTypeDetection:** Processes the uploaded photo to detect the skin type.
- After detection, the skin type is displayed back to the user.

Module 2: Recommendation Generation

Description: This involves sending skin type information for recommendation generation and delivering those recommendations to the user.

- **SkinTypeDetection:** Sends the detected skin type to Recommendations.

- **Recommendations:** Generates skincare product recommendations based on the detected skin type.
- The recommendations are provided to the user.

Module 3: External Interaction

Description: It focuses on user interaction with external websites for detailed product information.

- **User:** Clicks on a product link on the External Website.
- **External Website:** Provides detailed product information back to the user.

CODING

Model Building

The constructed model leverages the DenseNet169 pre-trained architecture to extract robust features from input images, which are then processed through custom fully connected layers for classification. The pre-trained layers are frozen to preserve their learned features, while additional dense layers are trained to adapt the model to the specific classification task. This approach ensures efficient and accurate classification, benefiting from the extensive feature learning of the DenseNet169 model. The model is compiled with the Adam optimizer and categorical cross-entropy loss function, making it suitable for multi-class classification problems

1. Importing Required Libraries

First, we import the necessary libraries:

```
# Model Building
from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.models import Sequential
```

- **Dropout:** A regularization technique to prevent overfitting by randomly setting a fraction of input units to zero during training.
- **BatchNormalization:** A technique to normalize the inputs of each layer to improve speed, performance, and stability.
- **Sequential:** A linear stack of layers in Keras.

2. Defining the Sequential Model

We initialize a sequential model:

```
# Define the Sequential model
keras_model = Sequential()
```

The **Sequential** model in Keras is a linear stack of layers.

3. Adding the Pre-trained DenseNet169 Model

We incorporate the DenseNet169 model pre-trained on ImageNet, excluding its top layers to allow customization:

```
# Add the pre-trained DenseNet169 model
pretrained_model = tf.keras.applications.DenseNet169(include_top=False,
                                                    input_shape=(640, 640, 3),
                                                    pooling='avg', weights='imagenet')
```

- **DenseNet169**: A pre-trained CNN model.
- **include_top=False**: Excludes the final classification layers.
- **input_shape=(640, 640, 3)**: Specifies the input dimensions.
- **pooling='avg'**: Applies global average pooling to reduce each feature map to a single value.
- **weights='imagenet'**: Uses weights pre-trained on the ImageNet dataset.

4. Freezing the Layers of the Pre-trained Model

To retain the learned features, we freeze the layers of the pre-trained model:

```
# Freeze the layers of the pre-trained model
for layer in pretrained_model.layers:
    layer.trainable = False
```

This ensures the pre-trained weights are not updated during training.

5. Adding the Pre-trained Model to the Sequential Model

We add the pre-trained DenseNet169 model to our sequential model:

```
keras_model.add(pretrained_model)
```

6. Flattening the Output

We flatten the output of the convolutional base to create a single long feature vector:

```
keras_model.add(Flatten())
```

7. Adding Fully Connected Layers with Dropout and Batch Normalization

We add custom fully connected layers with dropout and batch normalization to reduce overfitting and improve performance:

```
# Add a dense layer with dropout and batch normalization
keras_model.add(Dense(512, activation='relu'))
keras_model.add(BatchNormalization())
keras_model.add(Dropout(0.5))
```

```
# Add another dense layer
keras_model.add(Dense(256, activation='relu'))
keras_model.add(BatchNormalization())
keras_model.add(Dropout(0.5))
```

- **Dense:** Fully connected layer with ReLU activation.
- **BatchNormalization:** Normalizes the outputs of the dense layer.
- **Dropout:** Randomly drops 50% of the input units during training to prevent overfitting.

8. Adding the Final Classification Layer

The final layer is added for classification into three categories:

```
# Add the final classification layer
keras_model.add(Dense(3, activation='softmax'))
```

- **Dense(3, activation='softmax'):** Outputs a probability distribution across three classes using the softmax activation function.

9. Compiling the Model

The model is compiled with the Adam optimizer and categorical cross-entropy loss function:

```
# Compile the model
keras_model.compile(optimizer=Adam(learning_rate=0.001),
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])
```

- **optimizer=Adam(learning_rate=0.001):** Utilizes the Adam optimization algorithm with a learning rate of 0.001.
- **loss='categorical_crossentropy':** Employs categorical cross-entropy loss, suitable for multi-class classification tasks.
- **metrics=['accuracy']:** Tracks the accuracy of the model during training and evaluation.

TESTING

Testing is essential to ensure that software is of high quality, secure, reliable, and user-friendly. It helps identify and address issues early, saving time and costs, and ensuring the final product meets user expectations and regulatory standards. Without proper testing, software is likely to have defects that can lead to system failures, security breaches, and a poor user experience.

a) Unit Testing

Unit testing involves testing individual components or functions in isolation. In the context of your code, you can write unit tests for functions like `load_and_preprocess_image` and `predict_image_class`.

```
def test_load_and_preprocess_image():
    img_path = 'path_to_a_sample_image'
    img_array = load_and_preprocess_image(img_path)
    assert img_array.shape == (1, 640, 640, 3), "Image preprocessing failed"

def test_predict_image_class():
    img_path = 'path_to_a_sample_image'
    label, confidence = predict_image_class(keras_model, img_path, class_indices)
    assert label in class_indices.keys(), "Prediction label not in class indices"
    assert 0 <= confidence <= 1, "Prediction confidence out of bounds"

if __name__ == '__main__':
    pytest.main()
```

`test_load_and_preprocess_image`: Checks if the image preprocessing function produces the expected output shape.

`test_predict_image_class`: Verifies that the prediction function returns valid outputs within expected bounds.

b) Integration Testing

Integration testing checks if different parts of the system work together. For your model, you can ensure that the entire data pipeline, model training, and prediction process work together seamlessly.

```

import numpy as np
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from your_code_file import keras_model, class_indices

def test_integration():
    # Simulate the data generators
    test_datagen = ImageDataGenerator(rescale=1./255)
    test_generator = test_datagen.flow_from_directory(
        '/tmp/test/',
        target_size=(640, 640),
        batch_size=32,
        class_mode='categorical')

    # Evaluate the model using the test generator
    steps = math.ceil(test_generator.samples / test_generator.batch_size)
    test_loss, test_accuracy = keras_model.evaluate(test_generator, steps=steps)

    assert test_accuracy > 0.5, "Model accuracy is too low"

if __name__ == '__main__':
    pytest.main()

```

test_integration: Simulates the entire data pipeline, including data loading and model evaluation, to ensure that the model's accuracy meets a specified threshold.

c) Smoke Testing

Smoke testing is a quick check to see if the main functionalities of the code are working. You can perform a basic check to see if the model loads correctly, the data generators work, and a single prediction can be made.

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from your_code_file import keras_model, class_indices, load_and_preprocess_image,
predict_image_class

def test_smoke():
    # Check if model loads
    assert keras_model is not None, "Model did not load"

    # Check if data generator works
    test_datagen = ImageDataGenerator(rescale=1./255)
    test_generator = test_datagen.flow_from_directory(
        '/tmp/test/',
        target_size=(640, 640),

```

```

        batch_size=32,
        class_mode='categorical')
    assert test_generator.samples > 0, "Test generator did not find any samples"

    # Check if a prediction can be made
    img_path = 'path_to_a_sample_image'
    label, confidence = predict_image_class(keras_model, img_path, class_indices)
    assert label in class_indices.keys(), "Prediction failed"
    assert 0 <= confidence <= 1, "Prediction confidence out of bounds"

if __name__ == '__main__':
    pytest.main()

```

test_smoke: Checks if the model loads correctly, data generators work, and predictions can be made successfully.

d) Regression Testing

Regression testing ensures that new changes do not break existing functionality. You can maintain a set of test cases and validate that the output remains consistent over time.

```

import numpy as np
from your_code_file import keras_model, predict_image_class, class_indices

def test_regression():
    img_path = 'path_to_a_sample_image'
    expected_label = 'dry'
    expected_confidence_range = (0.8, 1.0)

    label, confidence = predict_image_class(keras_model, img_path, class_indices)
    assert label == expected_label, f"Expected {expected_label}, got {label}"
    assert expected_confidence_range[0] <= confidence <=
expected_confidence_range[1], \
        f"Expected confidence in range {expected_confidence_range}, got
{confidence}"

if __name__ == '__main__':
    pytest.main()

```

test_regression: Compares current predictions with expected outputs to detect any regressions in model behavior.

e) Black Box

Blackbox testing involves testing the system without looking at the internal code structure. You focus on inputs and expected outputs.

```
def test_blackbox():
    img_path_valid = 'path_to_a_valid_image'
    img_path_invalid = 'path_to_an_invalid_image'

    # Valid input
    label, confidence = predict_image_class(keras_model, img_path_valid,
class_indices)
    assert label in class_indices.keys(), "Prediction failed for valid input"
    assert 0 <= confidence <= 1, "Prediction confidence out of bounds for valid
input"

    # Invalid input
    try:
        label, confidence = predict_image_class(keras_model, img_path_invalid,
class_indices)
        assert False, "Expected an error for invalid input"
    except Exception as e:
        assert True, "Handled invalid input correctly"

if __name__ == '__main__':
    pytest.main()
```

test_blackbox: Tests valid and invalid inputs to ensure robust error handling and correct predictions.

f) White Box

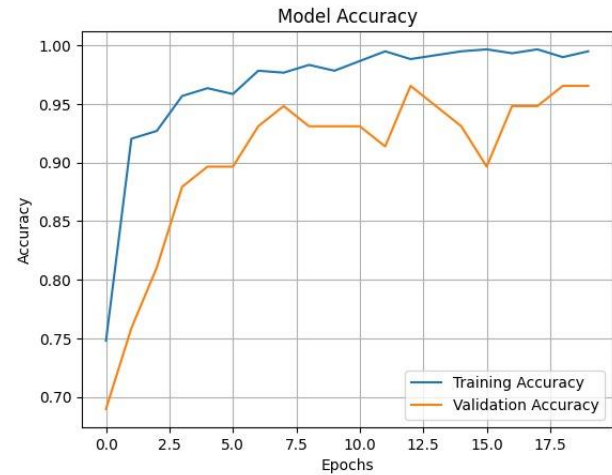
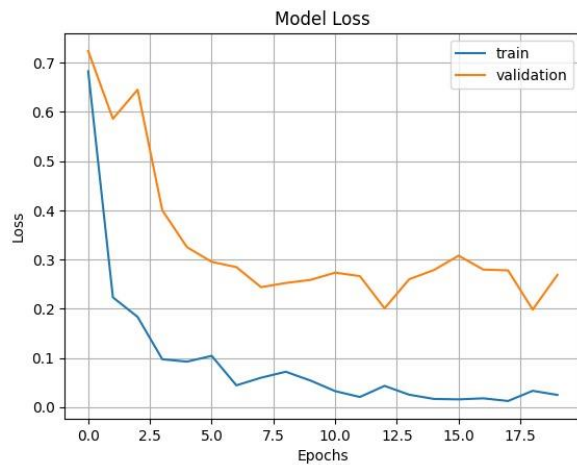
White box testing involves testing internal structures or workings of an application. For your code, you could create unit tests for internal methods and check for expected outcomes.

g) Performance Testing

Performance testing evaluates how well your model performs under certain conditions.

```
test_loss, test_acc = keras_model.evaluate(test_generator, verbose=2)

print("\nTest accuracy from evaluate:", test_acc)
```



1/1 - 2s - loss: 0.0421 - accuracy: 0.9643 - 2s/epoch - 2s/step

Test accuracy from evaluate: 0.9642857313156128

Product Name	Rating	Product Link	Description	Price
Love Beauty & Planet Skin Moisturising Combo At Nykaa, Best Beauty Products Online	5.050	https://www.nykaa.com/love-beauty-planet-skin-moisturising-combo/p/11771942?ptype=product&skuId=11771942&srsltid=Afm8Oopv_vsfqzcmiltp_dsSAVUj2u58J8s5QrPto_7bhlKnu7KSSQM	NaN	533
Physiogel Hypoallergenic Daily Moisture Therapy Cream	5.050	https://www.amazon.in/Physiogel-Hypoallergenic-Moisture-Sensitive-Provides/dp/B08D29N17Q?source=ps-sl-shoppingads-lpcontext&ref_=pplfs&psc=1&smid=A17X88J067GFJ2	Men's For Dry Skin - Cream	636.5
Dry Skin Care Routine Kit: Face Wash + Face Serum + Face Moisturiser Earth ...	4.927	https://earthrhythm.com/products/dry-skin-care-kit?variant=3947406422839&cy=INR&utm_medium=product_sync&utm_source=google&utm_content=sag_organic&utm_campaign=sag_organic&srsltid=Afm8OooknGecJ81skaylRTTtVfSpCODVnT4vyangduR_0YpnJlG1MSk	NaN	₹ 1,587.00
The Derma Co. x Dr.V Skin Renew Barrier Repair Peptide Moisturizer - Peptide ...	4.890	https://www.amazon.in/Derma-Moisturizer-Peptide-Moisturizes-Regeneration-Strengthens/dp/B0D1C19JBP?source=ps-sl-shoppingads-lpcontext&ref_=pplfs&psc=1&smid=A61C6NUFGVG57	Face - Men's - Vitamin C	499
Re'equil Dry Skin CTM Bundle	4.766	https://www.reequil.com/products/dry-skin-ctm-bundle?variant=32696053137540&cy=INR&utm_medium=product_sync&utm_source=google&utm_content=sag_organic&utm_campaign=sag_organic&srsltid=Afm8OorKZ3HCjOV0fGmoH13bIDW08o4bnP7XVdUj2QV-fidRl_FdnlroutGE	NaN	1055.55

1/1 [=====] - 0s 43ms/step
The image belongs to: dry with confidence: 0.8653



CONCLUSION

In conclusion, our research represents a significant advancement in the field of skin type classification and personalized skincare recommendations. By employing the DenseNet169 convolutional neural network architecture, we achieved an outstanding accuracy of 96.4%, surpassing previous models which typically reached around 89%. This accomplishment underscores the effectiveness of DenseNet169 in extracting intricate facial features essential for precise skin type classification. A key novelty of our approach lies in the pioneering use of DenseNet169 for skin classification, a model rarely explored in this context before our study. This decision proved instrumental as it not only enhanced accuracy but also highlighted the untapped potential of advanced deep learning architectures in dermatological applications.

Furthermore, our methodology integrated web scraping techniques to augment our dataset with 690 diverse facial images collected via Roboflow. This comprehensive dataset enabled robust training and validation of our model, ensuring its efficacy across various skin types and conditions. Our findings demonstrate the feasibility and benefits of leveraging cutting-edge technologies like DenseNet169 and web scraping in healthcare-related image analysis. Moving forward, these insights pave the way for future advancements in personalized skincare solutions, potentially revolutionizing how skincare professionals and individuals approach skincare regimens tailored to individual skin characteristics.

In conclusion, our research not only achieves superior accuracy but also establishes a new benchmark in utilizing advanced deep learning frameworks for dermatological applications, contributing significantly to both academic research and practical healthcare innovations.

REFERENCES

1. Lin, Ting-Yu, et al. "Facial Skincare Products' Recommendation with Computer Vision Technologies." *Electronics* 11.1 (2022): 143.
2. C. Chang et al., "Robust skin type classification using convolutional neural networks," 2018 13th IEEE Conference on Industrial Electronics and Applications (ICIEA), Wuhan, China, 2018, pp. 2011-2014
3. Kothari, D. Shah, T. Soni and S. Dhage, "Cosmetic Skin Type Classification Using CNN With Product Recommendation," *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, Kharagpur, India, 2021, pp1-6
4. J. Akshya, V. Mehra, M. Sundarrajan, P. T. Sri and M. D. Choudhry, "Efficient Net-based Expert System for Personalized Facial Skincare Recommendations," *2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, Madurai, India, 2023, pp. 1151-1156
5. Lee, Jinhee, et al. "Deep learning-based skin care product recommendation: A focus on cosmetic ingredient analysis and facial skin conditions." *Journal of Cosmetic Dermatology* (2024).